Efficient Proof Engines with Lambda-DAGs

ANONYMOUS AUTHOR(S)

In domains from pure math to program verification, arguably the most popular interactive proof assistants of today are all based on dependent type theory. They expose different *tactic languages* to guide automated generation of *proofs* in core logics. Their *proof engines* are the interface that underlies tactics: keeping track of open goals, detecting proof errors incrementally, and generating proof terms if desired. Use cases like classic math theorems and metatheory of core calculi have in common that intermediate goals never grow especially large, compared to use cases like verifying imperative programs that are thousands of lines long – the latter may produce proof goals with e.g. one free variable per assignment statement and one hypothesis per free variable explaining what we know about its value, and it is not practical instead to inline those free variables with their defining equations, as we lose sharing and grow goals exponentially. Therefore, if proof assistants are to break into the mainstream of routinely verifying important, large programs, core operations of proof engines must exhibit good asymptotics (effectively linear in goal size), *which is not the case for the popular type-theoretic proof assistants today.*

We propose a new style of proof engine, optimized for highly automated scripted proofs, that remedies this flaw. Our new engine is based on λ-DAGs, a technique for representing sets of terms in functional languages with high representation sharing. We evaluate this style with a prototype freestanding implementation that outputs Rocq proof terms, demonstrating enormous speedups in important cases motivated by verifying large programs.

1 Introduction

Proof assistants like Rocq [22] and Lean [17] expose high-level interfaces for writing mathematical proofs, where individual proof steps can be checked incrementally, without knowing the rest of the proof. Both systems can also translate those proof scripts into proof terms: expressions in dependently typed λ-calculi. Relatively simple algorithms can then revalidate the proof terms, regardless of which tactics (automation procedures) were used to produce them. We use the name *proof engine* for the proof-assistant component that provides key data structures for goals and partial terms, along with primitive incrementally checkable proof steps from which tactics are built (and assembles proof terms from tactic execution).

It is convenient to implement domain-specific proof automation using scripting languages like Rocq's Ltac [5]. However, all widely used proof engines exhibit serious performance challenges when running custom tactics against large goals. Fundamentally, the proof engine manages goals as contexts with free variables of known types and, more challengingly, existential variables that need to be determined by later proof steps, where each existential variable is tagged with the context it is valid in (not every free variable of the goal is legal to mention when instantiating every existential variable). Users of proof assistants who confine their attention to small goals, say in proving core-calculus metatheory rather than verifying significant software programs, may be surprised to learn that established proof engines expose key operations that run in time well-above linear in goal size. Natural automation methods will repeat these core operations in their tight inner loops. Further, proof-engine performance bottlenecks have forced developers of these systems to accept functionality limitations. For example, instantiating an existential variable in Rocq *defers critical scoping checks* until `Qed`, giving up immediate error detection to keep the running time under control.[1]

In some sense, the tactic-driven usage is also less demanding on the proof-engine data structures than the use case they originate from – elaboration and higher-order unification modulo conversion.

[1] https://github.com/rocq-prover/rocq/issues?q=label%3A%22wellknown%3A%20ltac%20variable%20bypasses%20typechecking%22

To contrast, tactic-driven program verifiers invoke the reduction rules of the underlying type theory infrequently but perform many more term constructions and proof steps than needed to internalize a handwritten preterm for the type theory. (Both kinds of proof-engine usage need efficient examination of terms and their types and contexts.) Informally, we have observed that elaboration and adjacent use cases are served decently well by current proof engines, with serious performance obstacles only arising with specific obscure programming patterns or generated code. Thus, we set out to build a new proof engine specifically for automated and tactic-driven verification. We are not currently recommending unification and elaboration algorithms be ported to it. The design we propose is straightforwardly interoperable with established proof-engine data structures on terms free of existential variables, so the two can coexist within a system.

We believe fundamental changes in data structures and algorithms are needed to allow natural proof scripting to coexist with good performance scaling, where a goal during verification of a realistic program may involve thousands of free variables. We propose a variant of λ-DAGs [20] as the center of a new style of proof engine, which we have prototyped as a freestanding C program, which is available as an anonymous supplement, and "scripted" with C procedures for now. Every operation the engine exposes runs in effectively linear time, in practice. Through benchmarking with small challenge problems, we demonstrate major speedups vs. Rocq and Lean. No changes to the trusted base of a proof assistant are required – our prototype outputs Rocq proof terms.

The advantages of this new style of proof engine come from careful control of sharing of subterms and tracking of their respective binding scopes and references. The richer invariants of the data structures we propose enable fast and incremental checking of proof scripts that were infeasible before. We hope to inspire the developers of all type-theoretic proof assistants to adopt these ideas, and there may also be applications in related domains like proof-term generation by SMT solvers (which also use sharing-optimized data structures internally).

The remainder of this introduction summarizes the data structure we suggest and the kinds of manipulations that are important to implement efficiently. Then we step back to explain our new engine in more detail, staged to explain the complications from management of $\forall$-style variable binders (section 2) and $\exists$-style pluggable holes (section 3). Our empirical evaluation (section 4) shows that even simple microbenchmarks, representative of tasks in verification of imperative programs, demonstrate meaningful asymptotic speedups over both Rocq and Lean. We close (section 5) with a survey of related work and broader discussion of the significance of our results.

1.1 λ-DAGs

The internal representation of terms in proof assistants can greatly impact automation performance. A *λ-DAG* [20] is a directed-acyclic-graph representation of λ-calculus terms, where subterms are referred to, allowing for sharing by reference. This sharing is crucial for both space and time efficiency, especially when manipulating large terms or performing repeated rewrites on terms with many references to the same subterm.

The details of the lambda-DAG approach are as follows. A λ-calculus term is represented as a rooted DAG, not a tree. **Variable references are globally unique** (and our `gensym` is `malloc`). In other words, any given variable will have a single node representing it, which is not necessarily the case for arbitrary subterms. Applications are represented as nodes with references to their function and argument subterms, while lambda abstractions have references to their bound variables and bodies. Finally, each node in the DAG maintains a list of its incoming references, that is, the other nodes that point to it (for example, all uses of a variable can be found efficiently).

Another central part of the lambda-DAG approach is the *single-DAG requirement*: that every subterm must only belong to one rooted DAG, and all terms manipulated by a tactic must be in the same DAG. Crucially, all terms in the DAG are well-typed, so there is no need to re-typecheck

them on use. The single-DAG assumption is used extensively to check whether a particular term can be used to instantiate a particular existential variable (i.e. whether the term is well-typed in a particular context). While past work on lambda-DAGs relies on this invariant for β-reduction without search or copying, we make due with simpler reduction.

$$\begin{array}{l}\text{let } y_1 := f\ x \text{ in}\\ \text{let } z_3 := g\ \underbrace{y}_{2}\ y \text{ in } z_3\end{array}$$

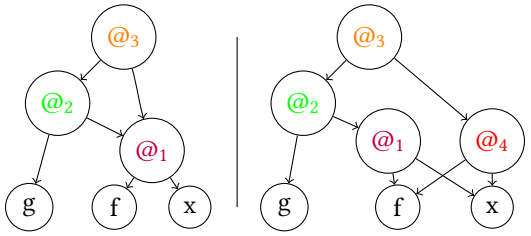


Fig. 1. Sharing in DAGs

For example, consider the following term, assuming f is a unary function and g is a binary function, both defined over some type:

```
let y := f x in
let z := g y y in z.
```

A lambda-DAG representation of this term with maximal sharing would create a subterm for the application of f to x. Then, when constructing the subterm for z, it would reference the existing subterm for y twice. Note that this lambda-DAG is not the only possible representation. As we show in Figure 1, the term could also be represented as a tree, where the subterm for y would be duplicated. This second variant corresponds to naive approaches that accept losing the sharing implied by the let-binder (which is logically valid as ζ-reduction but undesirable for performance). Our lambda-DAG-based tactics have been implemented carefully to *preserve sharing*.

As an extended example, consider the following term:

$$\text{add } (\text{add } (\text{add } x\ x)\ 0)\ (\text{add } (\text{add } x\ x)\ 0)\ 0. \tag{1}$$

Figure 2 shows a high-sharing lambda-DAG representation of this term. At the bottom of the figure are the variables `add`, `x`, and `0`. The *innermost* application $@_0$ points to `add` and `x`, representing the ordinary term `add x`. The next application $@_1$ points to $@_0$ and `x`, giving us the term `add x x`.

Continuing in this manner leads us to the application $@_3$. As specified by the user, this application is also named y_3, and some part of the proof engine may associate the variable name with this subterm. This naming allows us to continue building the expression by referencing $@_3$ in subsequent applications such as $@_4$ and $@_5$.

That we write `let` binders in source code corresponding to shared nodes in a λ-DAG is not a coincidence – the DAG nodes can be referenced everywhere some `let` binder with the same value could be, including type annotations of binders and other DAG nodes.

Another important consideration is that every subterm must be valid under some context. In our system, each variable node references the context in which it was defined, and the structure of each node reflects its associated context. For example, the constant `0` might be introduced in a context that first defines `nat` and `add`, followed by `0`. All application nodes in this example reside in

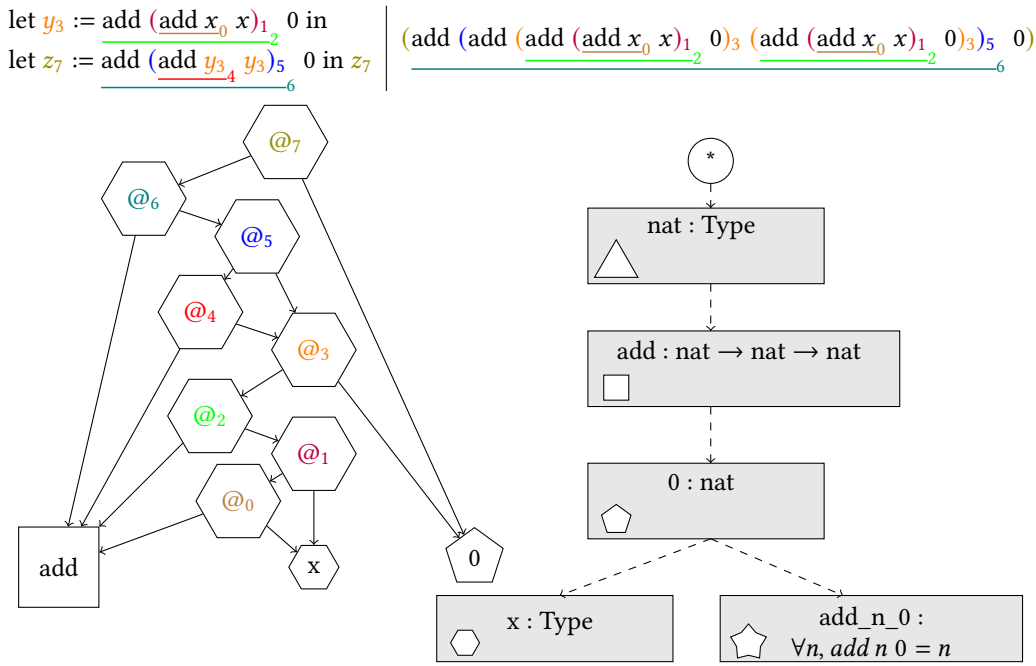


Fig. 2. DAG-structured example

the hexagonal context, which includes bindings for `nat`, `add`, `0`, and `x`. Section 2 explains in detail how our proof engine represents and uses contexts.

In our proof engine, every term is represented as a node in a DAG, with explicit tracking of variable binding scopes. This design enables efficient computation of scope intersections and fast substitution, which are essential for scalable proof automation. As we will show, this approach leads to significant performance improvements over traditional implementations.

1.2 Rewriting

Equational rewriting is a fundamental operation in proof assistants, allowing users to simplify expressions by applying lemmas that express equalities. SMT solvers also apply extensive rewriting within their e-graph engines. Our rewriter is a bottom-up recursive procedure inspired by Rocq's `rewrite_strat bottomup` and the rewriting strategy proposed by Gross et al. [13].

The rewrite procedure takes as input an expression to rewrite in and a lemma to rewrite with. It starts by recursively traversing to the leaves of the expression DAG and applies a rewrite-head function, which matches the input expression against the the lemma.

Our rewrite-head function attempts to instantiate the given lemma against the input expression using syntactic unification. This process may eliminate any leading quantifiers or simply confirm that the lemma applies directly. The result of this process is either a failure or a successful instantiation along with a list of any remaining goals.

If instantiation succeeds, we extract the type of the instantiated lemma and attempt to match the left-hand side of the resulting equality against the input expression. If they match, we return a `RewriteProof` object with the corresponding output expression, proof term, and list of goals. Otherwise, we return a trivial `RewriteProof`, again using `eq_refl`.

```
rewrite_head(ctx, expr, lemma) =
  match unify_and_instantiate(ctx, lemma, expr) with
  | None ⇒
    mk_proof(expr, expr, eq_refl(expr), [])
  | Some { lemma_inst, new_goals } ⇒
    let eq_ty = type_of(lemma_inst) in
    let rhs = rhs_of_eq(eq_ty) in
    mk_proof(expr, rhs, lemma_inst, new_goals)
```

In some cases, unification may not fully determine all arguments required to instantiate the lemma. Rather than failing outright, we introduce holes as placeholders for unknown expressions, similar to Rocq's existential variables. These allow the lemma to be applied partially, allowing rewriting to proceed.

The result of such a unification attempt includes not only the partially instantiated lemma but also a list of goals corresponding to the unresolved holes, which can be deferred and instantiated by later tactics. For example, consider the lemma:

$$\texttt{mult_0_l} : \forall x, 0 = 0 * x.$$

Suppose we want to rewrite the left-hand side of a goal $0 = 0 * y$ using this lemma. From the expression 0, we cannot immediately infer that x should be instantiated with y. However, the shape of the goal suggests that the lemma should apply, so we should not fail outright either. Instead, the engine introduces a hole $?x$ and applies the lemma as $\texttt{mult_0_l}\ ?x$, producing the statement $0 = 0 * ?x$. This partially instantiated equality is still usable in rewriting, and the remaining goal will eventually unify $?x$ with y. In this way, holes allow the engine to proceed with rewriting even with incomplete information, deferring certain decisions to later unification or tactic steps.

We can now describe the implementation of the rewriting procedure in more detail with focus on term and context manipulation, still eliding generic optimizations like memoization.

```
rewrite_app((func arg) as expr, lemma) =
  let rw_func = _rewrite(func, lemma) in
  let rw_arg = _rewrite(arg, lemma) in

  let mid =
    if nothing_rewritten(rw_func) && nothing_rewritten(rw_arg) then
      mk_proof(expr, expr, eq_refl(expr), [])
    else
      let merged = rw_func.goals ++ rw_arg.goals in
      let app    = mk_app(rw_func.expr, rw_arg.expr) in
      let cong   = build_app_cong(rw_func, rw_arg) in
      mk_proof(expr, app, cong, merged)
  in

  let rewritten_mid = rewrite_head(mid.expr, lemma) in

  if nothing_rewritten(rewritten_mid) then
    mk_proof(expr, mid.expr, mid.proof, mid.goals)
  else
    let merged = mid.goals ++ rewritten_mid.goals in
    let composed = eq_trans(mid, rewritten_mid) in
```

```
      mk_proof(expr, rewritten_mid.expr, composed, merged)
```

In the application case, we recursively rewrite the function and argument components. If either changes, we build a new application and produce a proof using the app_cong congruence rule. We then attempt to further simplify this new application via rewrite_head. If that succeeds, we chain the proofs using transitivity.

```
rewrite_lambda(ctx, λx:T. body as expr, lemma) =
  let inner_rw  = _rewrite(body, lemma) in
  let mid_e     = mk_lambda(y, inner_rw.expr[x → y]) in
  let mid_pf    = build_lambda_ext(mk_lambda(z, inner_rw.proof[x → z])) in
  let mid       = mk_proof(expr, mid_e, mid_pf, inner_rw.goals) in

  let rewritten_mid = rewrite_head(ctx, mid, lemma) in

  if nothing_rewritten(rewritten_mid) then
    mk_proof(expr, mid, mid_pf, inner_rw.goals)
  else
    let merged = mid.goals ++ rewritten_mid.goals in
    let composed = eq_trans(mid, rewritten_mid) in
    mk_proof(expr, rewritten_mid.expr, composed, merged)
```

In the λ-abstraction case, we recursively rewrite the body. The resulting equality is lifted to the full λ-abstraction via functional extensionality. While our techniques should support rewriting without extensionality, it is necessary for rewriting under binders. We then attempt a top-level rewrite using rewrite_head on the resulting abstraction.

```
rewrite_var(ctx, expr, lemma) =
  let rewritten = rewrite_head(ctx, expr, lemma) in
  if nothing_rewritten(rewritten) then
    mk_proof(expr, expr, eq_refl(expr), [])
  else
    rewritten
```

In the variable case, we simply attempt a single call to rewrite_head and return the result if successful; otherwise, we fall back to the reflexivity proof.

To avoid duplicate computation, the rewrite procedure incorporates memoization at each expression node. After rewriting, the resulting RewriteProof is stored directly on the expression as a memoized attribute, enabling subsequent rewrites of shared subexpressions to retrieve the result in constant time. Caching in this way allows the algorithm to maintain roughly linear time complexity in the size of the DAG.

At the end of a top-level call to rewrite, we recursively clear all memoized results, ensuring that subsequent rewrites, especially those using a different lemma, do not reuse stale data. Because the memoization cache does not track which lemma was used, reusing it across calls would be unsound. One alternative would be to use an external memoization cache keyed by both expression and lemma, allowing persistent reuse, but we opt for simplicity by clearing the memoization cache after each rewrite session.

Additionally, we implement a variant of rewrite capable of handling a user-specified list of lemmas rather than a single one. The main modification lies in the rewrite_head function, which now iteratively attempts to apply each lemma in the list until no further rewriting is possible. The

proofs are chained together using `eq_trans`, which allows for chaining multiple rewrites driven by different lemmas without requiring separate calls to `rewrite`.

The rewrite tactic described above operates directly on expressions, rewriting the left-hand sides of equalities in a forward manner. However, proof engines generally manipulate goals differently. Instead, the user typically specifies a goal to prove, and the engine generates a goal state or hole, with the expected type. In the case of equational rewriting, this goal takes the form of an equation. Each rewriting step then provides a proof that the current goal is equivalent to a new, simpler goal, effectively transforming the problem into a sequence of equivalence-preserving subgoals.

We provide `rewrite_transform`, which operates not on arbitrary expressions but directly on goals. Given a goal expression (a hole with an expected return type) and a rewrite lemma, this function attempts to transform the goal's expected type by applying the lemma. If successful, it produces two things: a new hole with the rewritten goal type, and a term that satisfies the original goal by invoking `eq_subst` with the rewrite proof and the new goal as an argument. The new goal may then be resolved independently. This tactic is more similar to `rewrite` in Rocq, which readers may be more familiar with.

$$\texttt{eq_subst} : \forall (P : \texttt{Prop})\ (Q : \texttt{Prop}),\ P = Q \to Q \to P.$$

In practice, a single lemma may not be sufficient to simplify the goal fully. For this reason, we also support `rewrites_transform`, which accepts an arbitrary list of lemmas and applies them exhaustively. Each application rewrites the current goal type and updates the proof term accordingly, chaining substitutions via nested uses of `eq_subst`. This process continues until no lemma results in further simplification. These tactics provide a bridge between our rewriting engine and goal-directed proof construction.

2 Handling Binders

2.1 Contexts

Contexts in our system play a central role in tracking the assumptions under which terms are defined. They serve a purpose similar to Rocq's `Section` mechanism, but unlike in Rocq, contexts here are designed to be constructed and manipulated programmatically. Although our prototype supports automatic context management, we will explain how to use contexts deliberately to take full advantage of the proof engine.

At the base of the system is the empty context, which exists as a singleton. Larger contexts are built incrementally by extending existing contexts with new variable bindings. Each context node holds the type of a variable, a pointer to its parent context, and the length of the context. That is, a context $\Gamma = [x_1{:}A_1], [x_2{:}A_2], \ldots, [x_n{:}A_n]$ is represented as a chain of `Context` structs, each pointing back to its predecessor.

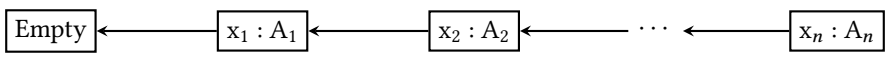


Fig. 3. A context $\Gamma = [x_1{:}A_1], [x_2{:}A_2], \ldots, [x_n{:}A_n]$ represented as a right-growing chain of variable bindings, each pointing to its parent.

The type of a variable being added to a context must itself be well-formed under the parent context. Variables themselves are pointer-unique, so a single variable expression can appear in multiple contexts, provided that its type remains valid.

2.2 The Context Tree

In Figure 2, we show an example of an expression alongside the global context tree. The variables `add`, `x`, and `0` are defined in various contexts, as illustrated. However, the variable `add_n_0` is defined in a context that does not include `x`. Recall the term in Expression 1, which we labeled as $@_7$ in Figure 2. If we were to construct an expression that references both the application $@_7$ and the lemma `add_n_0`, we would first need to create a new context that includes both variables.

While the global shape of all contexts in the system may resemble a tree, the most efficient pattern is for expressions to share a common context spine. This design allows validity checks between contexts to be reduced to just parent traversal and pointer comparisons, which becomes especially important when dealing with existential variables, which have often been a pain point in performance of proof engines [15].

To take advantage of this structure, we aim to construct goals with this linear organization in mind. When preparing to prove a goal, we begin by separately defining the axioms that will be required. We then build a single context by inserting sets of bindings in an order that reflects their dependencies, in a process loosely resembling dependency management. Concretely, when proving a goal involving `add_r_0` : $\forall n,\ n + 0 = n$, we might start by axiomatizing the natural numbers as a type, addition as a binary operator, and separately defining equality and its reflexivity rule. In terms of contexts, this structure is illustrated in Figure 4.

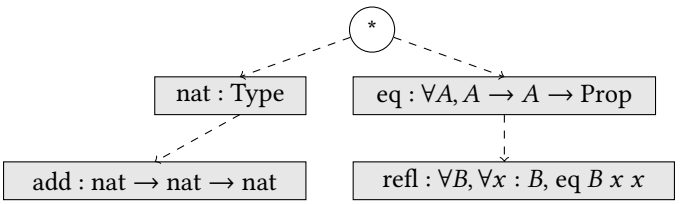


Fig. 4. Global context tree after axiomatizing natural numbers, addition, and equality.

To axiomatize the lemma `add_r_0`, we must ensure that its context includes all bindings present in the tree shown in Figure 4. We do so by extending one branch of the context tree with the other, producing a new, unified context in which we can define the lemma. After doing so, the context tree may appear as in Figure 5. Following this manipulation, we can begin defining the goal in this extended context, adhering to the same incremental pattern.

3 Handling Evars

3.1 Holes

We implement a version of existential variables (evars), which we refer to as holes, to represent unknown expressions within the proof engine. A hole is defined by an expected type and a context representing the environment in which the hole is valid. Holes are introduced mostly as the results of partially successful unification. Our implementation does not support backtracking, so there is no way to uninstantiate a hole (it could be added using general persistent-data-structures techniques, but following Lean, we question the utility vs. the performance cost).

The primary operation on holes is instantiating them with concrete terms. This operation is subject to three constraints: (1) the term must have a type that matches the hole's expected type, (2) the context in which the term is defined must be valid within the context of the hole, and (3) the term filling the hole cannot contain a reference to the hole itself. The first condition can be

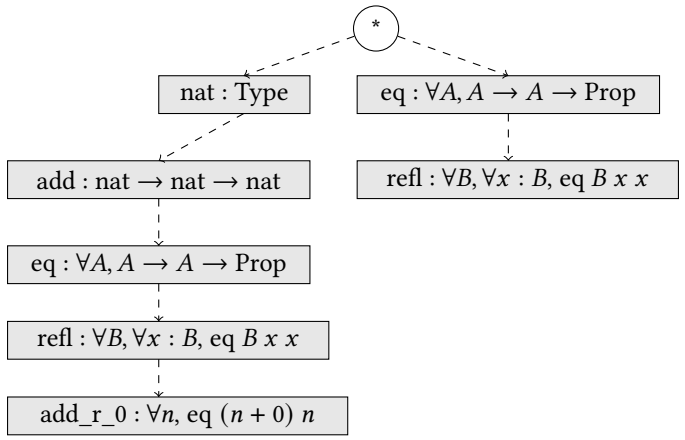


Fig. 5. Global context tree after axiomatizing the lemma `add_r_0`.

verified easily via congruence. The second and third conditions, involving context inclusion and occurs-checking respectively, are more challenging to verify efficiently.

In general, the term's context may be any valid subset of the hole's context. However, our proof engine optimizes for the case when the candidate term's context is a prefix of the hole's defining context. In this situation, the substitution can be performed in linear time relative to the size of the context. Overall in this case, the complexity of filling a hole is linear in both the size of the context and the size of the type's DAG.

One powerful feature of our internal representation is that it allows efficient propagation of the substitution of a term into a hole. When a suitable term is substituted into a hole, we do not need to traverse the entire DAG to find all occurrences of the hole. Instead, we can directly replace the hole's reference in the DAG with the new term.

For example, suppose we are attempting to apply the lemma

$$\texttt{add_r_0} : \forall n,\ n + 0 = n$$

to the goal

$$(x + 1) + 0 = x + 1.$$

Assume that in this case, we are taking advantage of the λ-DAG structure by having both occurrences of $x + 1$ be the same object. We might start by making a copy of the type of the lemma and instantiating it with a hole:

$$?n + 0 = ?n \tag{2}$$

where the type of the hole is a natural number and the defining context is the same as the context of the goal. Suppose we have successfully unified the hole with the term $x + 1$. In this case, there would be two occurrences of the hole in the DAG: one in the left-hand side of the goal and one in the right-hand side.

Naive handling of the shared term could be problematic in several ways. Firstly, if we naively attempted to construct a new term by substituting the hole with the term $x + 1$, we might end up with a term that has two copies of $x + 1$, which would be correct but loses the sharing implicitly present in the DAG. Secondly, if we were to traverse the entire DAG to find all occurrences of the hole, we face an avoidable linear time overhead, which becomes significant if the DAG is large.

Instead, we can directly replace the hole's reference in the DAG with the new term. As long as we guarantee that the term being substituted is valid in the context of the hole, we can perform this operation without having to redo the typechecking.

3.2 Substitution and Reduction

We currently take a simple approach to substitution, implementing simultaneous substitution over terms with binders using the standard, recursive algorithm that α-renames binders as it descends through the term. We must rename binders because, as the λ-DAG represents them with pointers, it is critical that the binders in the result of substitution are distinct from those in the input term. We write $e[t_1/x_1, ..., t_n/x_n]$ to stand for the simultaneous substitution of each x_i in e with t_i. While most substitutions we encounter, for example the ones produced by β-reduction, begin as a single substitution $e[t/x]$, implementing simultaneous substitution makes handling α-renaming straightforward. To understand why, consider the following example substitution that occurs while typechecking the term `eq_refl nat`:

```
(∀ x : A, eq A x x) [nat/A]
= ∀ x' : nat, (eq A x x) [nat/A,x'/x]
= ∀ x' : nat, eq nat x' x'
```

When we apply the substitution replacing `A` with `nat` to the ∀, we construct a new binder x' with the type `nat` as per the results of applying the substitution to `A`. As we proceed in processing the return type, we must now additionally replace each `x` with `x'`, which we can easily accomplish with simultaneous substitutions.

We chose this design for our initial development in the interest of ensuring correct implementation. It is deliberately simple while working well with our pointer-based context representation. Additionally, it efficiently represents the sharing that naturally occurs in β-reduction since even when a variable occurs many times, each instance gets replaced with a pointer to the same substituted term. However, it does incur overhead: each substitution may involve a full traversal of the expression, and each descent into a binder introduces new allocations and list extensions. In particular, for large terms with deeply nested binders, or when substituting small expressions into many contexts, performance can suffer. Additionally, it does not always preserve sharing already present in the term acted on by the substitution. Despite the potential for further improvements here, our experimental results in section 4 demonstrate that even this simple implementation is sufficient to improve performance greatly over the state of the art with our engine.

An alternative approach would be to adapt the method described by Shivers and Wand [20]. That work assumes a simple untyped λ-calculus and performs substitution efficiently by making use of the uplinks in the λ-DAG representation, without needing to account for the types of internal binders. Under this model, substitution does not require traversing the entire DAG; instead, variable references can be updated directly in-place. Crucially, correctness relies on the single-DAG requirement: each subterm must belong to exactly one root DAG, and node identifiers within a DAG act as globally unique names. In our setting, we could attempt a similar strategy by first making a copy of the λ-expression, refreshing the binders, and then using uplinks to perform substitution within this new copy. However, the details of that last step are subtle. To avoid typing errors, one must carefully manage binder freshness and the types of newly introduced variables. In our prototype, we found it complex to implement this approach correctly due to these subtle issues around binder freshness and type correctness. Nonetheless, it remains a promising direction for future asymptotic improvements.

3.3 Proof-Engine Operations

Our proof engine provides a trusted set of operations that manipulate terms and contexts, which we now describe. These operations, listed in Figure 6, form the boundary between tactics and the proof engine: they are the only functions permitted to modify the internal λ-DAG directly, and each is designed to be locally sound. Tactics such as `rewrite` are implemented entirely in terms of these operations and basic data structures.

```
(* Constructors with context arguments *)
mk_var_wc(name, type, context) → expr
mk_lambda_wc(bound_var, body, context) →
expr
mk_app_wc(func, arg, context) → expr
mk_forall_wc(bound_var, body, context) →
expr

(* Constructors with automatic
   context management *)
mk_type() → expr
mk_prop() → expr
mk_var(name, type) → expr
mk_lambda(bound_var, body) → expr
mk_app(func, arg) → expr
mk_forall(bound_var, body) → expr
```

```
(* Context management *)
mk_empty_context() → context
is_empty_context(ctx) → bool
context_extend(ctx, var) → context
is_ancestor_context(ctxA, ctxB) → bool
find_in_context(ctx, var) → context

(* Unification and congruence checking *)
unify(lemma, goal) →
option<instantiated_lemma x list<goal>>
congruence(a, b) → proof

(* Hole management *)
mk_hole(type, context) → expr
fill_hole(hole, term) → unit
```

Fig. 6. Trusted proof-engine interface and related operations.

There are two common themes in our proof-engine operations. First, each operation is locally sound: each operation will fail if poor inputs are provided, such as providing a term that is not well-typed in the provided context. Second, each operation can be used in a way that is efficient. Most of these operations are designed to be linear with respect to part of the input, but there exist pathological cases where the time complexity can degrade to quadratic. We briefly describe the operations below and their associated time complexities.

The first set of operations are constructors for various expression types, such as variables, λ-abstractions, and applications. The `mk_var_wc` function constructs a variable with a specified name, type, and context. The `mk_lambda_wc` and `mk_app_wc` functions create λ-abstractions and applications, respectively, while `mk_forall_wc` constructs universal quantifications. These constructors ensure that the resulting expressions are well-formed under the provided contexts; otherwise they fail. Each of these constructors performs the relevant typechecking to make sure that the resulting expression is well-formed. Each of these operations can take linear time in the size of the input context when following certain usage patterns. Additionally, forming an application requires a congruence check and substitution for the forming of the type, which add to the time complexity.

Concretely, suppose we have the expression $\lambda(n : \texttt{nat}).\ n + 1$ closed under the context $\Gamma = [\texttt{nat} : \texttt{Type}]*$ and we wish to form the application of this term to some variable $a : \texttt{nat}$. In reality, these are not symbolic expressions but pointers to C structures. The first step is to create the C structure representing $a : \texttt{nat}$ using the `mk_var_wc` constructor. The key to good performance is selection of an appropriate context. We create a using the context Γ, and the reason for this choice will become apparent momentarily. Next, we create a context that extends Γ with a using the context-extension

operation. Since the context of a shares a spine with Γ (in fact, it is Γ), this operation runs in linear time with respect to the size of the context. Finally, to create the application of the λ-abstraction to a, we provide the same Γ context. Again, since both the function and argument's contexts share a spine with Γ (in fact, are Γ), this operation completes in linear time.

The constructors in the second set automatically manage the context for the created expressions. These functions are convenient for quickly building terms without needing to specify the context manually each time. However, they still ensure that the resulting expressions are valid under a suitable context. Additionally, they are not strictly *required* for our proof engine and can be quite slow when used excessively.

The context-management operations allow users to create an empty context, check if a context is empty, insert a variable type into a context, check if one context is an ancestor of another, and find a variable in a context. The first two operations are implemented in constant time. The third operation, `context_extend`, extends the context with a new variable, and its time complexity can vary greatly depending on the surrounding usage.

In ideal circumstances, the `context_extend` operation takes linear time in the size of the context, but it can degrade to quadratic time depending on the context used to define the variable. In practice, this quadratic factor is not typically an issue. The `is_ancestor_context` and `find_in_context` operations can be performed in linear time by traversing pointers.

The `unify` function attempts to unify a lemma with a goal, returning an option type that contains the instantiated lemma and any remaining goals if successful. The time complexity of this operation is linear in the product of the number of binders of the lemma and the sum of the sizes of the λ-DAGs of the types of the lemma and the expression. The `congruence` function checks whether two expressions are α-congruent and is linear in the size of the λ-DAGs being compared.

Finally, the hole-management operations allow users to work with holes in the proof engine. The `mk_hole` operation is similar to the first set of constructors. The `fill_hole` operation performs the substitution of a term into the hole. Since any terms that reference the hole have already been typechecked, we only need to check that the term is well-typed in the context of the hole and succeed or fail accordingly. Usually, this operation is linear in the size of the context and the size of the type of the hole. However, if the context in which the term is defined is not a prefix of the hole's context, as explained in Section 2, the time complexity can degrade to quadratic.

4 Evaluation

In this section, we will evaluate the performance of our proof engine against existing proof engines by examining experimental asymptotics. We consider four main questions:

(1) What performance do we get from Rocq and Lean?
(2) With conventional methods, how good can we expect the performance to be?
(3) How well does our new design (which we call `MEngine`) perform?
(4) What is a reasonable lower bound?

4.1 Basic Rewriting

We start by looking at a very basic example where we simplify an expression, under no binders, using rewrite rules. Consider the following expression:

$$f\,(f\,(\cdots f\,(a)\cdots))$$

where a has some type and f is a unary function over that type. For simplicity, assume that type is `nat`, so a : `nat` and f : `nat` $\to$ `nat`. If a is a fixed point for the function f and we have a theorem expressing the equality, we can expect to rewrite away all the occurrences of f to be left with a.

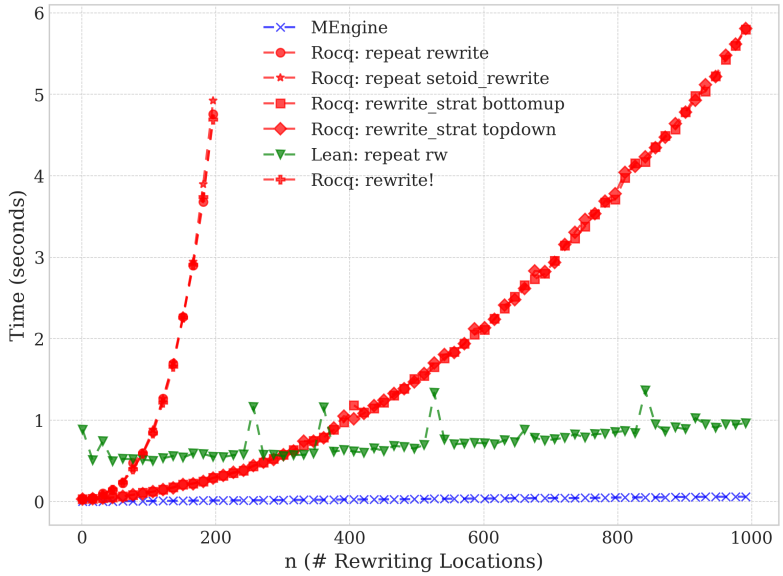


Fig. 7. Execution time to prove `eq nat f(f(...f(a)..)) a` in different proof engines.

Note that in this example, there are $\Theta(n)$ rewriting locations, so we might expect both the proof-term size and time complexity to grow linearly. As shown in Figure 7, the time complexity is **superlinear** in Rocq! We used the following code to generate the desired expression and rewrite back to a:

```
Fixpoint repeat_f (n : Datatypes.nat) : nat :=
  match n with
  | 0 ⇒ a
  | S n' ⇒ f (repeat_f n')
  end.

Goal eq nat (repeat_f __N__) (a).
Proof.
  cbn [repeat_f].
  Time (repeat rewrite eq_fa_a).
  (* or (rewrite_strat topdown eq_fa_a), or (rewrite! eq_fa_a). *)
  apply eq_refl.
Qed.
```

Not only does our approach outperform Rocq (which is expected, since our approach was written in C), it outperforms *asymptotically*, achieving $\Theta(n)$ time performance. This performance also seems asymptotically optimal, since there are $\Theta(n)$ rewriting locations.

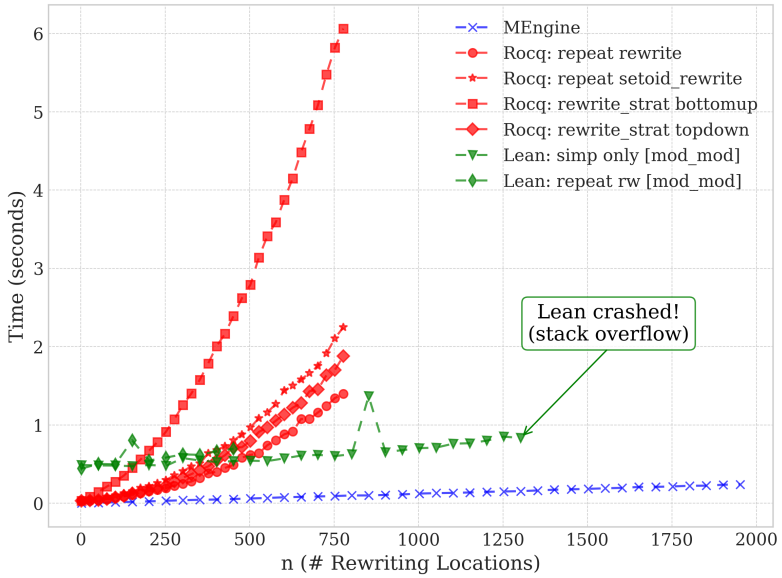


Fig. 8. Execution time to prove $(((a \bmod m) \bmod m) \cdots) = a \bmod m$ in different proof engines.

Next, consider the well-known benchmark problem of rewriting away chained or repeated modulos, minimized from term-simplification challenges faced by Fiat Cryptography [9].

$$\begin{aligned}&\text{let } x_1 = a \bmod m\\&\text{let } x_2 = x_1 \bmod m\\&\quad\vdots\\&\text{let } x_n = x_{n-1} \bmod m\\&\text{in } x_n = a \bmod m,\end{aligned}$$

In this case, there are also $\Theta(n)$ rewriting locations. Surprisingly, or perhaps unsurprisingly, Rocq's performance on this problem is also superlinear, as shown in Figure 8, while we might have expected to achieve linear performance. Lean initially performs well, but we were unable to benchmark for $n > 500$ due to a reported stack overflow error (`ulimit -s unlimited`, `maxHeartbeat`, and `maxRecDepth` did not help).

4.2 Rewriting under Binders

Now we consider the performance of rewriting under binders. In Rocq, rewriting under binders is a well-known pain point, and it is often difficult to achieve good performance in such cases, even when automatic inlining of let-binders is prevented.

The benchmark we focus on now will be an extension of the previous one: consider a function f with n arguments of the same type, returning one output of that type. We can nest this function m times recursively using let-bindings. In other words, we define a sequence of m variables where each is built by applying f to n copies of the previous term. We then apply rewrites to the expression using the following lemma:

$$\texttt{f_x0_n__x0} : f(x_0, x_0, \ldots, x_0) = x_0.$$

Note that in this example, maintaining the let-bindings is crucial. If the term expanded fully, then we would have $\Theta(n^m)$ rewriting locations. However, if allowed to rewrite under let-binders, we could potentially achieve performance that is linear in n and m instead. Example Rocq code where $n := 2$ and $m := 3$ is shown below:

```
Context (x0 : nat) (f : nat → nat → nat) (f_n_x0 : f x0 x0 = x0).
Goal let x1 := f x0 x0 in let x2 := f x1 x1 in let x3 := f x2 x2 in x3 = x0.
Proof. simpl. rewrite! f_n_x0. apply eq_refl. Qed.
```

The results of running this example in Rocq, Lean, and our proof engine are shown in Figure 9. In our approach, we achieve roughly linear time in both n and m, as expected. Rocq's default behavior is to **inline let-bindings silently**. Thus the execution time of this benchmark is superlinear in both n and m when using `rewrite!` and `setoid_rewrite`, and slightly better when using `rewrite_strat topdown` and `rewrite_strat bottomup`.

Next, consider another similar example using let-bindings from Gross [15]:

$$\texttt{let } v_1 := v_0 + v_0 + 0 \texttt{ in } \quad \ldots \quad \texttt{let } v_n := v_{n-1} + v_{n-1} + 0 \texttt{ in } \quad v_n + v_n + 0$$

where the goal is to rewrite away all occurrences of $+0$, of which there are $\Theta(n)$. Note that in this example, maintaining the let-bindings is crucial. Performing ζ-reduction on this term to get rid of the let-bindings causes the number of rewrite locations to spike up to $\Theta(2^n)$. To escape Rocq's default behavior of inlining let-in expressions, the folowing setup using a definition `Let_In` is adapted from Gross [15]:

```
Definition Let_In {A P} (x : A) (f : forall a : A, P a) : P x
:= let y := x in f y.
Lemma Let_In_def : @Let_In = fun A P x f ⇒ let y := x in f y.
Proof. reflexivity. Qed.
Global Strategy 100 [Let_In].
Hint Opaque Let_In : rewrite.
Global Instance Let_In_nd_Proper {A P} :
  Proper (eq ==> pointwise_relation _ eq ==> eq P) (@Let_In A (fun _ ⇒ P)).
```

This approach can be helpful in certain scenarios but does not completely alleviate the performance issues.[2] Furthermore, it is cumbersome and takes an unjustifiedly large amount of effort to understand and apply. In both approaches, the user must use the less-performant `setoid_rewrite` tactic to remove $+0$'s, since the `rewrite` tactic is unable to rewrite under binders.

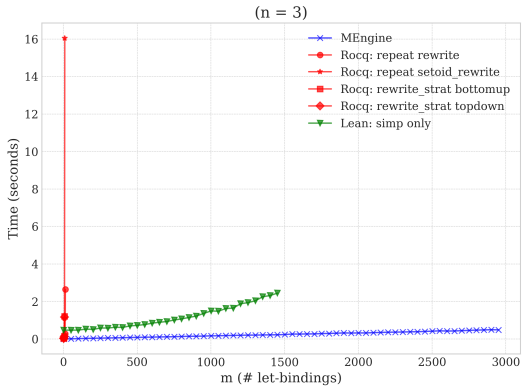
In our proof engine, we use native sharing to avoid wasting time redoing work. In Lean, we use the built-in `simp only` tactic, which performs simplification by bottom-up rewriting.

Figure 10 shows the performance of Rocq, Lean, and our proof engine on this example. Using `simp only` in Lean performs very well in comparison to both methods in Rocq and the `repeat rw` in Lean. Our approach asymptotically outperforms both, achieving near linear-time performance.

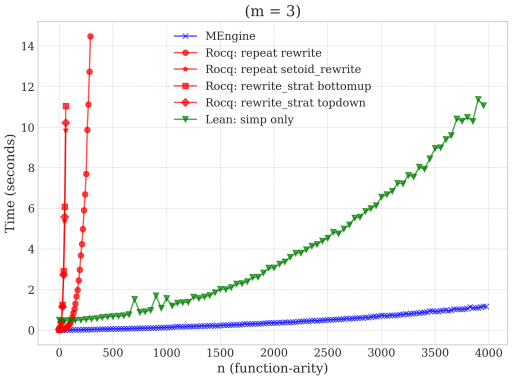
4.3 Symbolic Execution

The next example is inspired by Bedrock2 to show how we compare in real-world use cases. Consider the following language, which is a simplified subset of the language implemented in the Bedrock2 project [7]. That project encountered a number of recurring performance and reliability issues in Rocq, including slow or unpredictable unification, tactic failures, and limitations in automation when working with parameterized code. These challenges often forced developers to write workarounds in Ltac or avoid certain proof patterns entirely. While not the focus of that

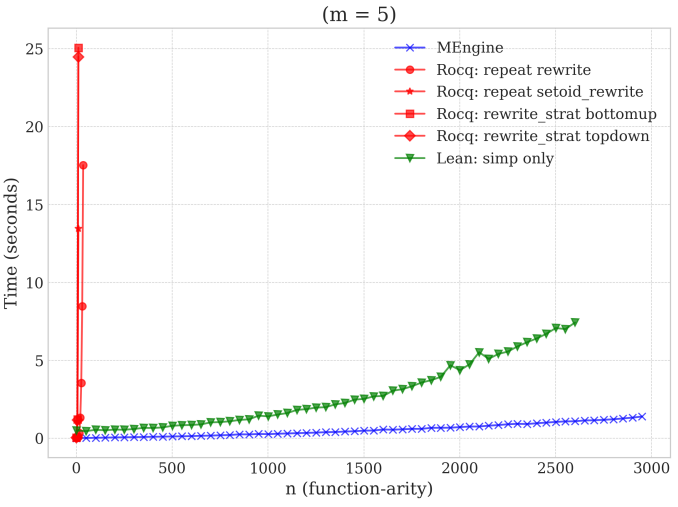
[2] See: https://github.com/rocq-prover/rocq/issues/12510



(a) Fixed $n = 3$



(b) Fixed $m = 3$



(c) Fixed $m = 5$

Fig. 9. Rewriting performance comparison between Rocq and our proof engine across parameter settings.

paper, the underlying theme reinforces our claim that many of the bottlenecks present in proof assistants are not inherent to the logic.

We now consider a concrete example using this subset of the Bedrock2 language. The following snippet defines a small imperative language in Rocq:

```
Context (bopname : Set) (bopname_add : bopname) (bopname_sub : bopname).
Inductive expr := literal (_:Z) | var (_:string) | op (_:bopname) (_ _:expr).
Context (cmd : Set) (skip : cmd) (set : string → expr → cmd)
        (seq : cmd → cmd → cmd) (input : string → cmd).
```

We write `seq` as `;` as usual, for example to double an input:

```
input a;  set a (bopname_add (expr_var a) (expr_var a))
```

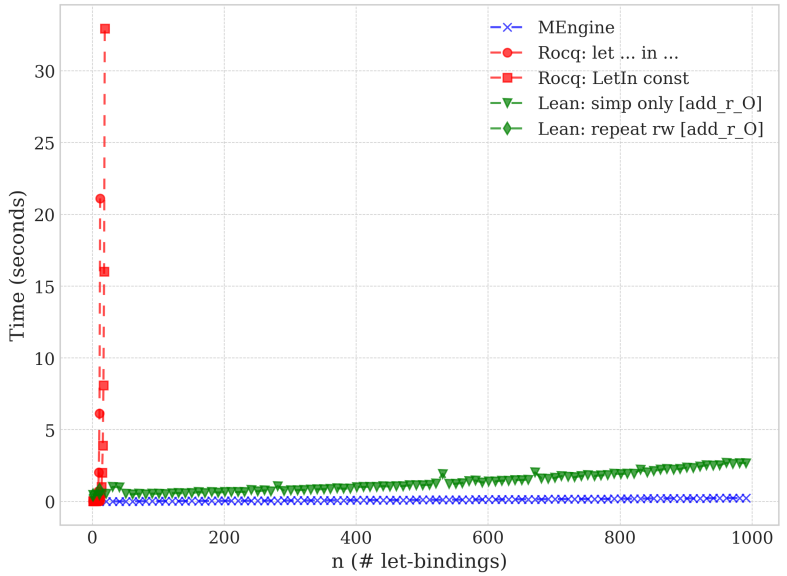


Fig. 10. Execution time of removing +0's from equation 1.

Following the approach used in the Bedrock2 project, we adopt an *omnisemantics* model [3], which relates a command and an initial state (memory, local variables, and trace) to a set of possible outcomes (a postcondition), rather than to a single deterministic result. This model is well-suited for reasoning about programs that exhibit undefined behavior or nondeterminism, since a single derivation characterizes all possible executions of a command:

```
Variable exec : cmd → list IOEvent → finmap word byte → finmap string word →
          (list IOEvent → finmap word byte → finmap string word → Prop) → Prop.
```

An `IOEvent` represents an input or output event, such as the result of an external function call or user input. The `partial_map word byte` type represents memory, while the local variables are represented by the type `partial_map string word`. Together, these three components (trace, memory, and locals) capture the full state of the program. The final argument to the command is a predicate on the final state. Altogether, the execution semantics assert that a command, when run from a given state, must satisfy the given postcondition. The following are the rules defining the `exec` relation:

Exec-Skip

$$\frac{Q\ t\ m\ l}{\text{exec skip}\ t\ m\ l\ Q}$$

Exec-Set

$$\frac{\text{eval_expr}\ m\ l\ e = \text{Some}\ v \qquad Q\ t\ m\ (\text{finmap_put}\ l\ x\ v)}{\text{exec}\ (\text{set}\ x\ e)\ t\ m\ l\ Q}$$

Exec-Seq

$$\frac{\text{exec}\ c_1\ t\ m\ l\ (\lambda t'.\ \lambda m'.\ \lambda l'.\ \text{exec}\ c_2\ t'\ m'\ l'\ Q)}{\text{exec}\ (\text{seq}\ c_1\ c_2)\ t\ m\ l\ Q}$$

Exec-Input

$$\frac{\forall v.\ Q\ (\text{IOEvent_IN}\ v :: t)\ m\ (\text{finmap_put}\ l\ lhs\ v)}{\text{exec}\ (\text{input}\ lhs)\ t\ m\ l\ Q}$$

To automate proofs of programs built with these semantics, we wrote a tactic and a test program. The generated program consists of a sequence of operations structured as follows:

input x;
set t (expr_var x);
set t (expr_op bopname_add (expr_var t) (expr_var t));
set t (expr_op bopname_sub (expr_var t) (expr_var x));
⋮ (repeated n times)
set t (expr_op bopname_add (expr_var t) (expr_var t));
set t (expr_op bopname_sub (expr_var t) (expr_var x));

To prove the correctness of goals about these generated programs, we implemented a `repeat_exec` tactic in Rocq, Lean, and `MEngine`, to automate reasoning about straightline code. The tactic is designed to solve goals in the following manner:

```
Goal ∀ (m : finmap word byte) (l : finmap string word),
    exec (generated_cmd __N__ a b) [] m l
      (fun (t' : list IOEvent) (m' : finmap word byte) (l' : finmap string word) ⇒
        m' = m ∧ ∃ v : word, t' = [IOEvent_IN v] ∧ finmap_get word l' a = Some v).
Proof.
    unfold generated_cmd; cbn [repeated_cmds]; intros.
    apply exec_seq; apply exec_input; intros.
    Time repeat_exec.
Admitted.
```

Here, `__N__` is a placeholder for the desired number of repetitions. Note that Bedrock2 similarly implements a tactic for reasoning about straightline code, and this design is loosely inspired by it. The implementation of this tactic in Rocq is as follows:

```
Ltac solve_eq_by_rewriting :=
  let step := first
    [ rewrite finmap_get_put_same | rewrite finmap_get_put_diff by apply b_neq_a
    | rewrite binop_add_to_word_sub | rewrite binop_add_to_word_add ] in
simpl; repeat step; apply eq_refl.

Ltac repeat_exec := repeat match goal with
  | [ |- exec (seq _ _) _ _ _ _ ] ⇒ apply exec_seq
  | [ |- exec (set _ _) _ _ _ _ ] ⇒ try rewrite finmap_put_put_same;
        eapply exec_set; try solve_eq_by_rewriting
  | [ |- exec skip _ _ _ _ ] ⇒ apply exec_skip           end.
```

In `MEngine`, we implement a similar straightline tactic for solving `exec` goals, though with important differences due to our core language design. Since `MEngine` does not directly implement inductive types, we added a small set of primitives `FixExpression` and `MatchExprExpression`, along with normalization procedures. Additionally, `MEngine`'s proof strategy includes solving the postcondition after the `repeat_exec` step. For reasons that are apparent in Figure 11, we do not bother solving the postcondition in Rocq and Lean, and we instead focus on the amount of time it takes to get to the postcondition.

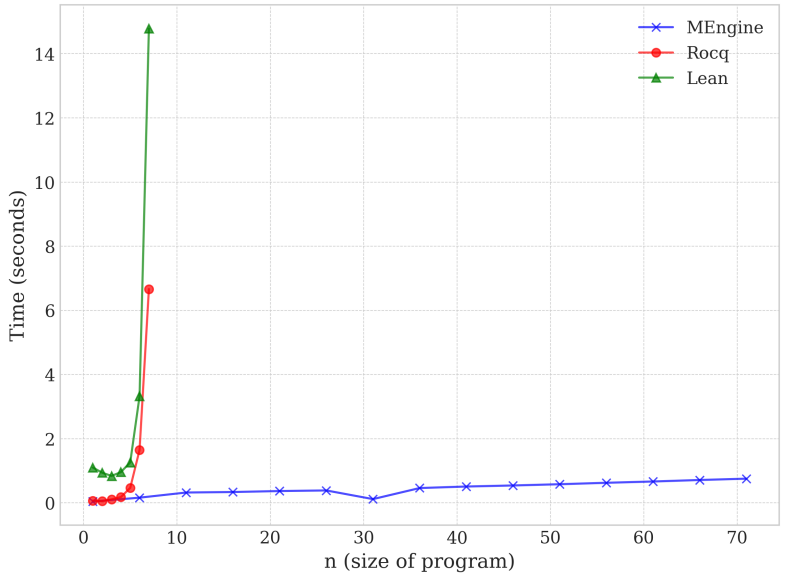


Fig. 11. Results of solving the Bedrock2-like symbolic-execution goals in Rocq, Lean, and `MEngine`.

The results of running this benchmark in Rocq, Lean, and `MEngine` for different values of n are shown in Figure 11. As shown, after $n := 4$, Rocq and Lean were spending an unreasonable amount of time to reach the postcondition, while our approach took only a few seconds.

5 Related Work

Proof engines for dependent type theory. The proof engine of Rocq got a significant overhaul in version 8.4, developed by Arnaud Spiwack.[3] We are not aware of a detailed writeup or performance evaluation for this work, but the focus seems to have been functionality rather than performance: refining existential variables using tactics, applying tactics to several goals simultaneously, reordering goals, etc. It is this upgrade that equipped Rocq with evars whose instances carry context substitutions, as opposed to metavariables that do not.

Lean's tactic and metaprogramming system is described in [6] along with a monoid-cancellation benchmark comparing to MirrorShard [16] on 100-clause closed terms. Lean metavariables carry contexts like Rocq's evars, but instances of Lean's metavariables do not carry context subtitutions. Instead, Lean abstracts an expression containing metavariables by creating an *application* of a new metavariable to the variable the previous metavariable was abstracted over[4] (i.e., normal λ-abstraction and application are reused for metavariable context substitutions). Lean4 further uses in-place mutation for data structures that are used linearly, in both object code and tactics [6]. However, there does not appear to be a discipline to ensure linear usage of metavariable or context objects, and the linearity requirement seems to be directly at odds with heavy use of backpointers in λ-DAGs.

A number of software-verification papers note performance challenges we attribute to the Rocq proof engine [4, 8, 9]. We are not aware of comparable publications in Lean; anecdotal reports

[3] https://vbgl.github.io/coq-84
[4] https://leanprover-community.github.io/mathlib4_docs/Lean/MetavarContext.html

in personal communication suggest that this shortage is due to performance issues preventing software-verification developments from reaching comparable scale.

There is a separate consideration of performance of mathematical abstractions in proof assistants, especially during elaboration and unification rather than tactic programming, e.g. in the Mathematical Components [10] library or HoTT category theory [12], and Lean mathlib [24]. A commonality with the challenges tackled in this paper is that maintaining sharing of subterms is a central consideration.

Reflection-based approaches. A great deal of effort has been invested in *avoiding* the general-purpose proof engines of proof assistants, moving as much of the proof process as possible into special-purpose evaluation mechanisms instead. Perhaps the most elaborate framework of this kind is MirrorShard [16], which implements an entire other tactic language as an executable procedure inside Rocq. Great performance improvements are demonstrated but for only a simply typed subset of the proof assistant's language and a significantly restricted tactic-programming API. Later systems such as the rewriter of [14] use another layer of metaprogramming to reduce the toil of creating reflective versions of relevant definitions and lemmas, but the generality limitations remain.

Critically, the notions of evars supported by these systems are much less flexible than those of any general-purpose proof engine discussed here. Indeed, it would be surprising to see a speedup from implementing *the same* proof engine inside a proof assistant's logic when compared to an implementation in a system-native imperative language.

Separately, mathematics developments also use reflection to improve proof performance [2, 11, 23]. It is not immediately clear whether the proof-engine improvements proposed here would be helpful there.

LCF-style systems. The proof objects constructed by tactics in LCF-style systems are superficially similar to λ-DAG proofs in this work, associating each theorem with the context under which it holds and using sharing of the ambient ML-like language for terms. However, these systems do not maintain a single-DAG invariant, and thus, similarly to Rocq, reusing a term constructed in one context in another requires a full traversal of it.

While we are not aware of broad study of proof-engine performance in LCF-style systems, the specific case of symbolic evaluation (a special case of rewriting) has received considerable attention. These results are quite relevant, as conversion tactics in LCF-style systems follow a high-level structure similar to our rewriting procedure. For example, symbolically evaluating the CakeML compiler on itself in HOL4 takes about 30 hours despite substantial optimization and downscoping [21]. Complete evaluation of primes up to 2048 using the conversion procedure took 24s in HOL4, 83s in HOL Light, and 132s in Isabelle in 2012 [1]. Comparison of reflection-based partial evaluation in HOL and Rocq can be found in [14].

E-graphs. The λ-DAG representation actually resembles a restricted, acyclic e-graph representation [18, 25]. While λ-DAGs do not track equality, their uplinks resemble the structure of the parent tracking that e-graphs use to compute congruence closure. Given the close connection, it seems worth investigating whether our procedures might generalize from λ-DAGs to e-graphs. In the other direction, recent work on improving sharing between α-equivalent subterms in e-graphs may have applications in our setting [19].

References

[1] Oskar Abrahamsson and Magnus O. Myreen. 2023. Fast, Verified Computation for Candle. In *14th International Conference on Interactive Theorem Proving (ITP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 268),*

Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 4:1–4:17. doi:10.4230/LIPIcs.ITP.2023.4

[2] Frédéric Besson. 2006. Fast reflexive arithmetic tactics the linear case and beyond. In *International Workshop on Types for Proofs and Programs*. Springer, 48–62.

[3] Arthur Charguéraud, Adam Chlipala, Andres Erbsen, and Samuel Gruetter. 2023. Omnisemantics: Smooth Handling of Nondeterminism. *ACM Trans. Program. Lang. Syst.* 45, 1, Article 5 (March 2023), 43 pages. doi:10.1145/3579834

[4] Adam Chlipala. 2011. Mostly-automated verification of low-level programs in computational separation logic. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 234–245. doi:10.1145/1993498.1993526

[5] David Delahaye. 2000. A Tactic Language for the System Coq. In *Proc. LPAR*.

[6] Gabriel Ebner, Sebastian Ullrich, Jared Roesch, Jeremy Avigad, and Leonardo de Moura. 2017. A metaprogramming framework for formal verification. *Proc. ACM Program. Lang.* 1, ICFP, Article 34 (Aug. 2017), 29 pages. doi:10.1145/3110278

[7] Andres Erbsen. 2023. *Foundational Integration Verification of Diverse Software and Hardware Components*. Ph.D. thesis. Massachusetts Institute of Technology. Advisor(s) Adam Chlipala. https://hdl.handle.net/1721.1/150216

[8] Andres Erbsen, Samuel Gruetter, Joonwon Choi, Clark Wood, and Adam Chlipala. 2021. Integration verification across software and hardware for a simple embedded system. 604–619. doi:10.1145/3453483.3454065

[9] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. 2019. Simple High-Level Code for Cryptographic Arithmetic - With Proofs, Without Compromises. In *2019 IEEE Symposium on Security and Privacy (SP)*. 1202–1219. doi:10.1109/SP.2019.00005

[10] François Garillot, Georges Gonthier, Assia Mahboubi, and Laurence Rideau. 2009. Packaging Mathematical Structures. In *Proceedings of the 22nd International Conference on Theorem Proving in Higher Order Logics* (Munich, Germany) *(TPHOLs '09)*. Springer-Verlag, Berlin, Heidelberg, 327–342. doi:10.1007/978-3-642-03359-9_23

[11] Georges Gonthier. 2008. Formal Proof—The Four-Color Theorem. *Notices of the American Mathematical Society* 55, 11 (2008), 1382–1393. https://www.ams.org/notices/200811/tx081101382p.pdf

[12] Jason Gross, Adam Chlipala, and David I. Spivak. 2014. Experience Implementing a Performant Category-Theory Library in Coq. In *Interactive Theorem Proving*, Gerwin Klein and Ruben Gamboa (Eds.). Springer International Publishing, Cham, 275–291.

[13] Jason Gross, Andres Erbsen, Jade Philipoom, Rajashree Agrawal, and Adam Chlipala. 2024. Towards a Scalable Proof Engine: A Performant Prototype Rewriting Primitive for Coq. *Journal of Automated Reasoning* 68, 3 (Aug. 2024). doi:10.1007/s10817-024-09705-6

[14] Jason Gross, Andres Erbsen, Jade Philipoom, Miraya Poddar-Agrawal, and Adam Chlipala. 2022. Accelerating Verified-Compiler Development with a Verified Rewriting Engine. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPICS.ITP.2022.17

[15] Jason S. Gross. 2021. *Performance Engineering of Proof-Based Software Systems at Scale*. Ph.D. Thesis. Massachusetts Institute of Technology, Department of Electrical Engineering and Computer Science, Cambridge, MA. Advisor(s) Adam Chlipala. https://hdl.handle.net/1721.1/130763

[16] Gregory Malecha, Adam Chlipala, and Thomas Braibant. 2014. Compositional Computational Reflection. In *Interactive Theorem Proving*, Gerwin Klein and Ruben Gamboa (Eds.). Springer International Publishing, Cham, 374–389.

[17] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 625–635. doi:10.1007/978-3-030-79876-5_37

[18] Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (April 1980), 356–364. doi:10.1145/322186.322198

[19] Rudi Schneider, Marcus Rossel, Amir Shaikhha, Andrés Goens, Thomas Kundefinedhler, and Michel Steuwer. 2025. Slotted E-Graphs: First-Class Support for (Bound) Variables in E-Graphs. *Proc. ACM Program. Lang.* 9, PLDI, Article 223 (June 2025), 23 pages. doi:10.1145/3729326

[20] Olin Shivers and Mitchell Wand. 2010. Bottom-up Beta-reduction: Uplinks and Lambda-DAGs. *Fundam. Inf.* 103, 1–4 (Jan. 2010), 247–287.

[21] Yong Kiam Tan, Magnus O. Myreen, Ramana Kumar, Anthony Fox, Scott Owens, and Michael Norrish. 2016. A new verified compiler backend for CakeML. *SIGPLAN Not.* 51, 9 (Sept. 2016), 60–73. doi:10.1145/3022670.2951924

[22] The Rocq Development Team. 2025. *The Rocq Prover*. doi:10.5281/zenodo.17473943

[23] Laurent Théry and Guillaume Hanrot. 2007. Primality proving with elliptic curves. In *Proceedings of the 20th International Conference on Theorem Proving in Higher Order Logics* (Kaiserslautern, Germany) *(TPHOLs'07)*. Springer-Verlag, Berlin, Heidelberg, 319–333.

[24] Eric Wieser. 2023. Multiple-Inheritance Hazards in Dependently-Typed Algebraic Hierarchies. In *Intelligent Computer Mathematics*, Catherine Dubois and Manfred Kerber (Eds.). Springer Nature Switzerland, Cham, 222–236. https://arxiv.org/pdf/2306.00617

[25] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. Egg: Fast and Extensible Equality Saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (jan 2021), 29 pages. doi:10.1145/3434304